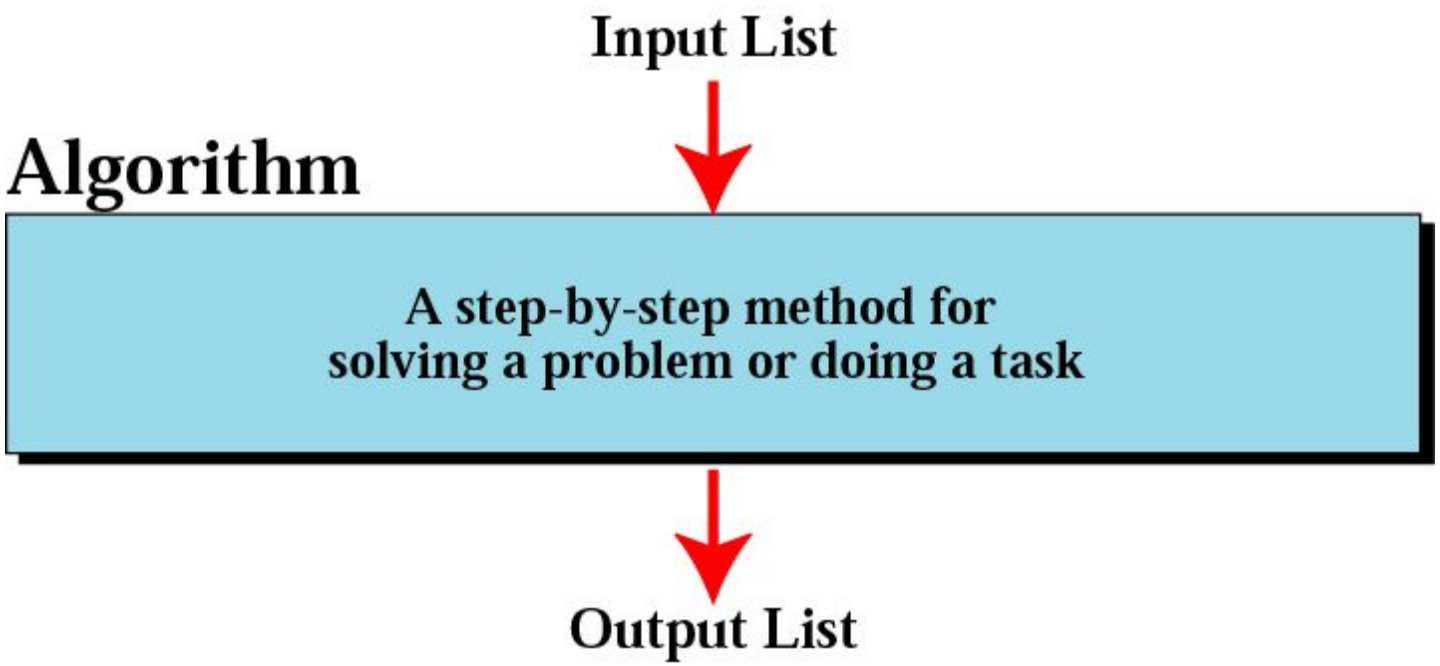
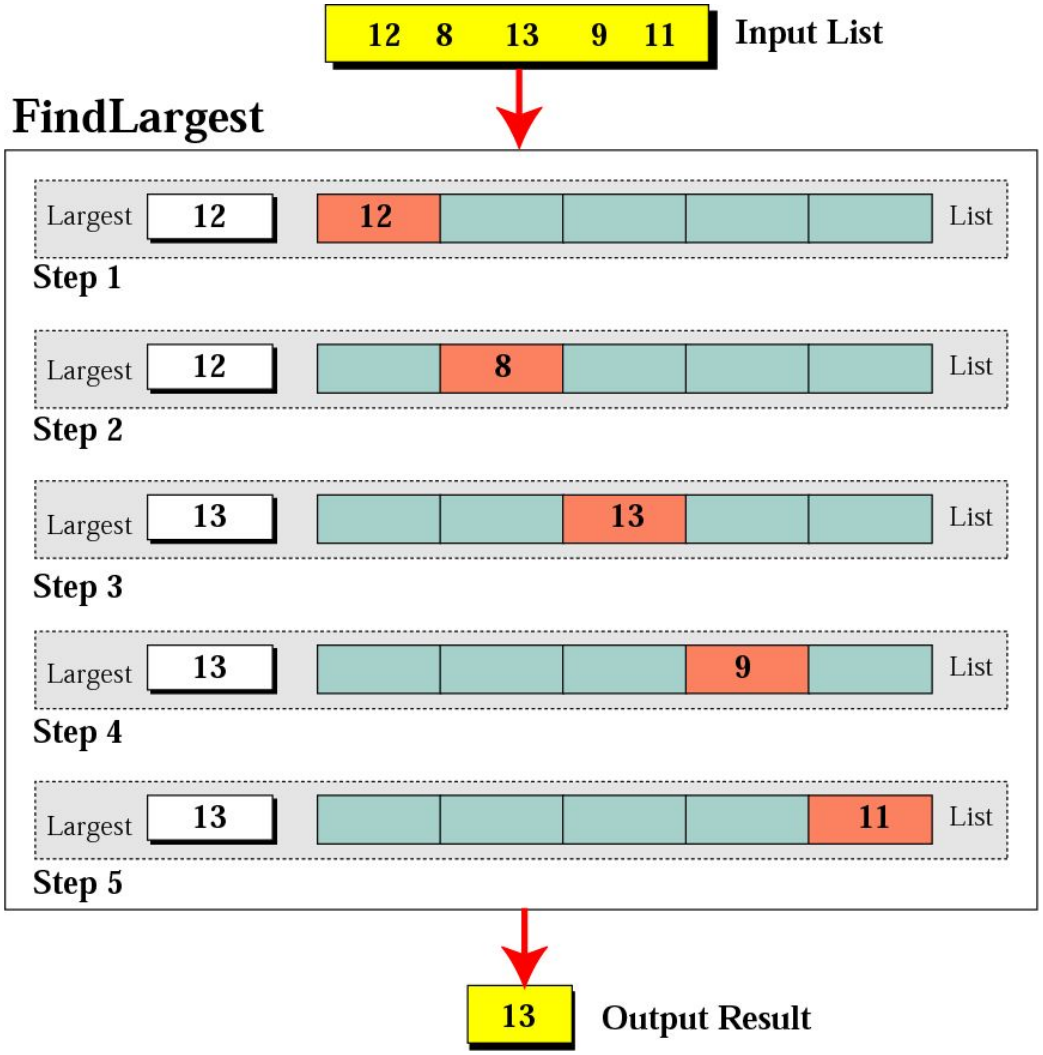
Algorithms

CONCEPT

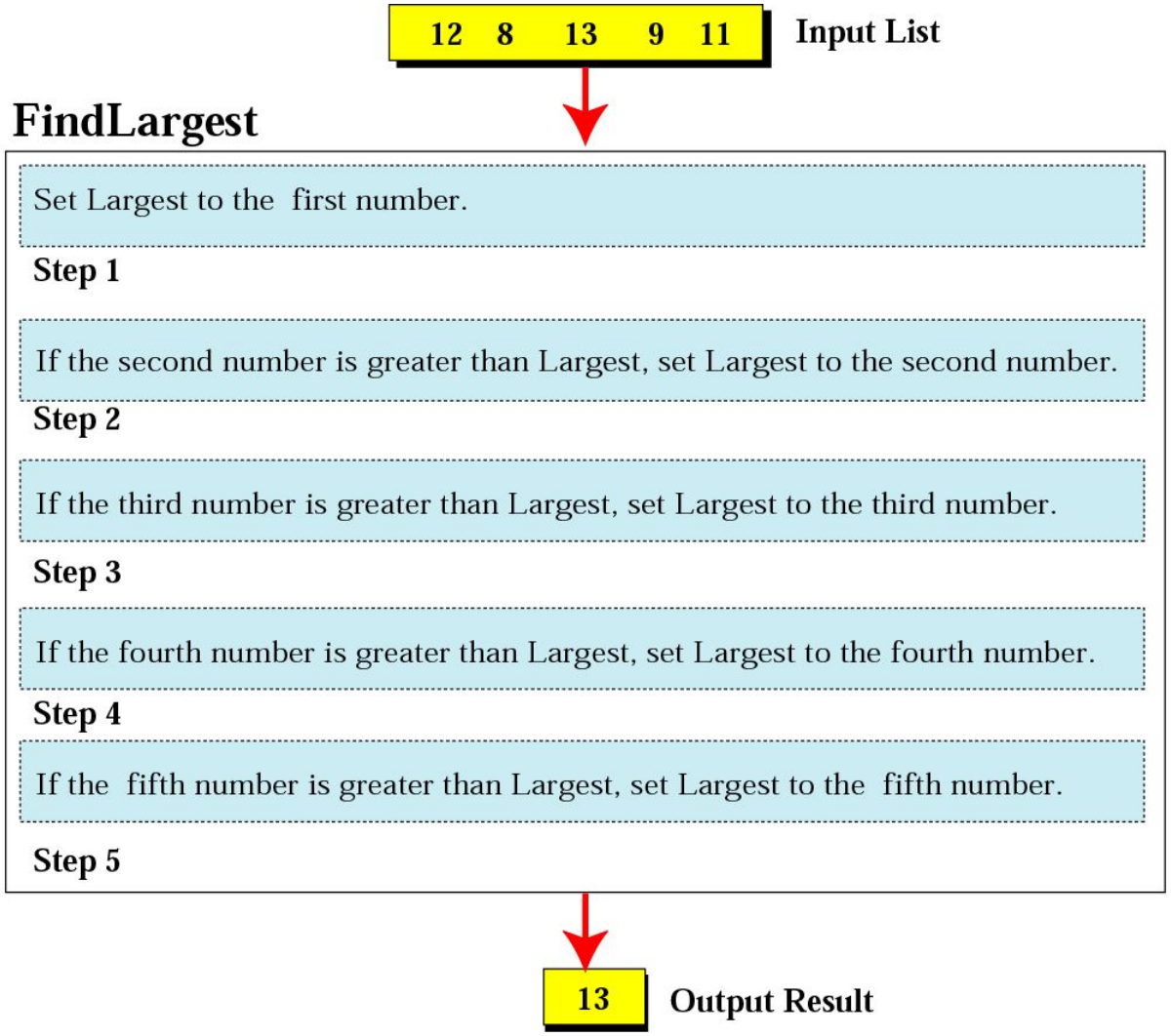
Informal definition of an algorithm used in a computer



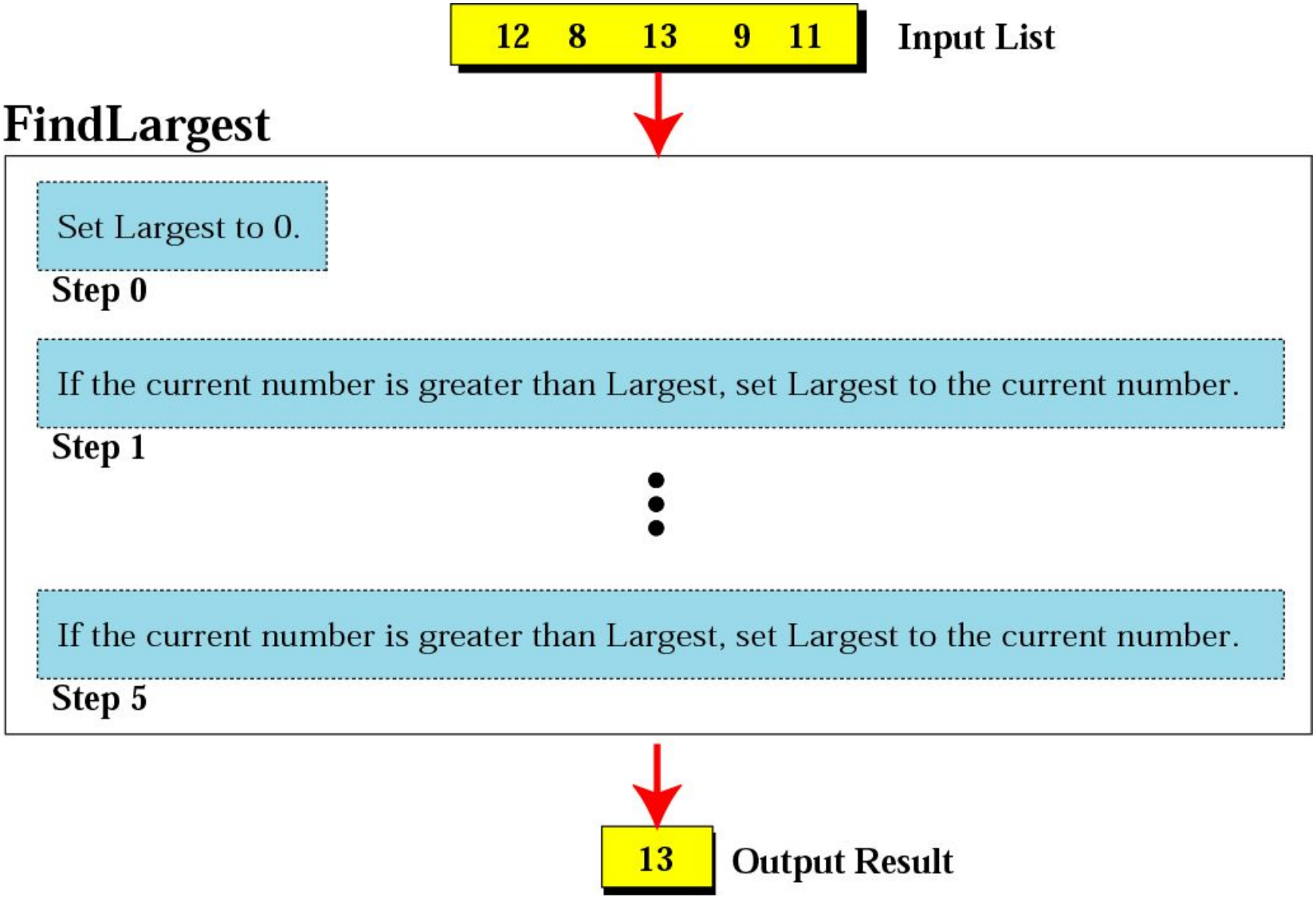
Finding the largest integer among five integers



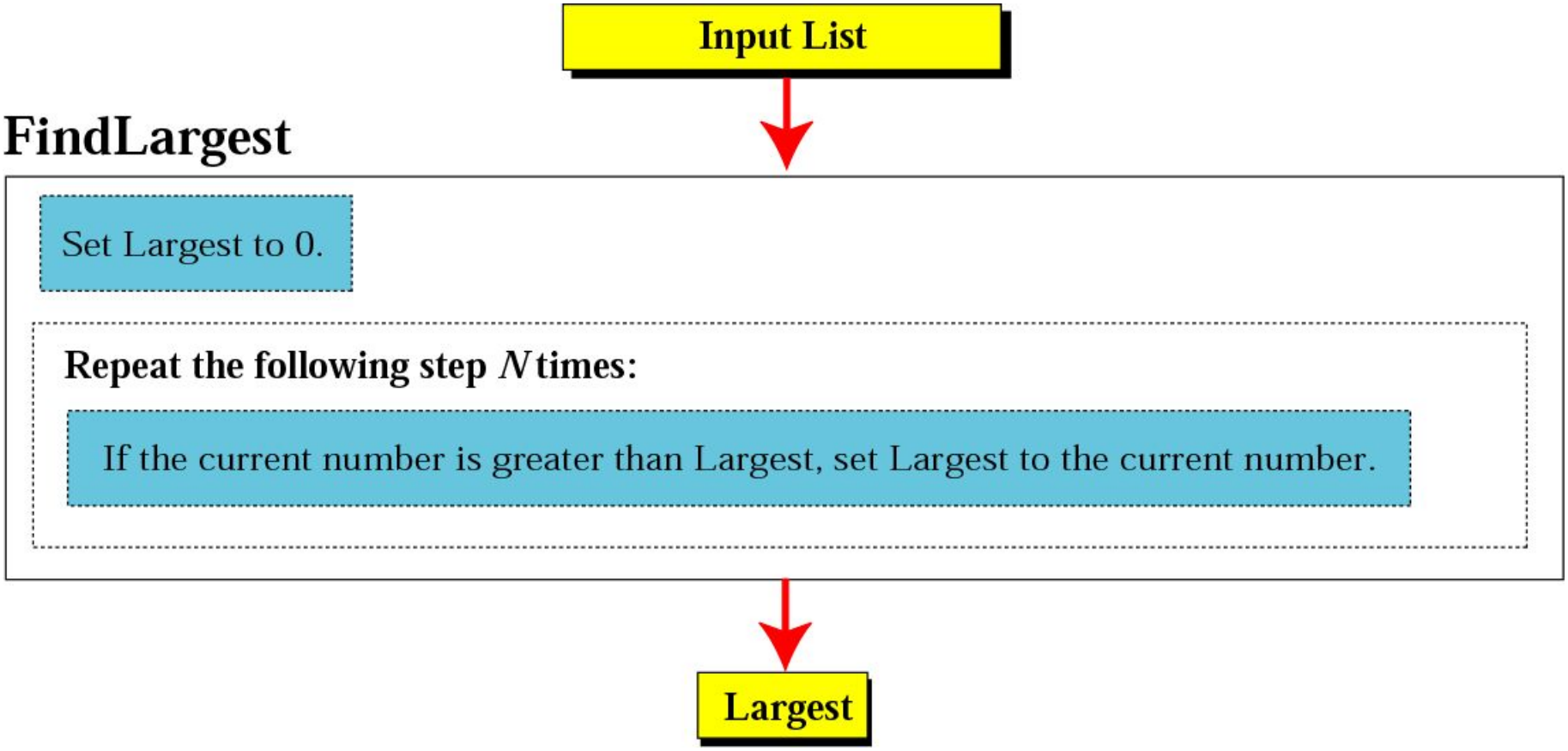
Defining actions in FindLargest algorithm



FindLargest refined



Generalization of FindLargest



THREE CONSTRUCTS

Three constructs

do action 1
do action 2
. . .
. . .
do action *n*

a. Sequence

if a condition is true,
then
do a series of actions
else
do another series of actions

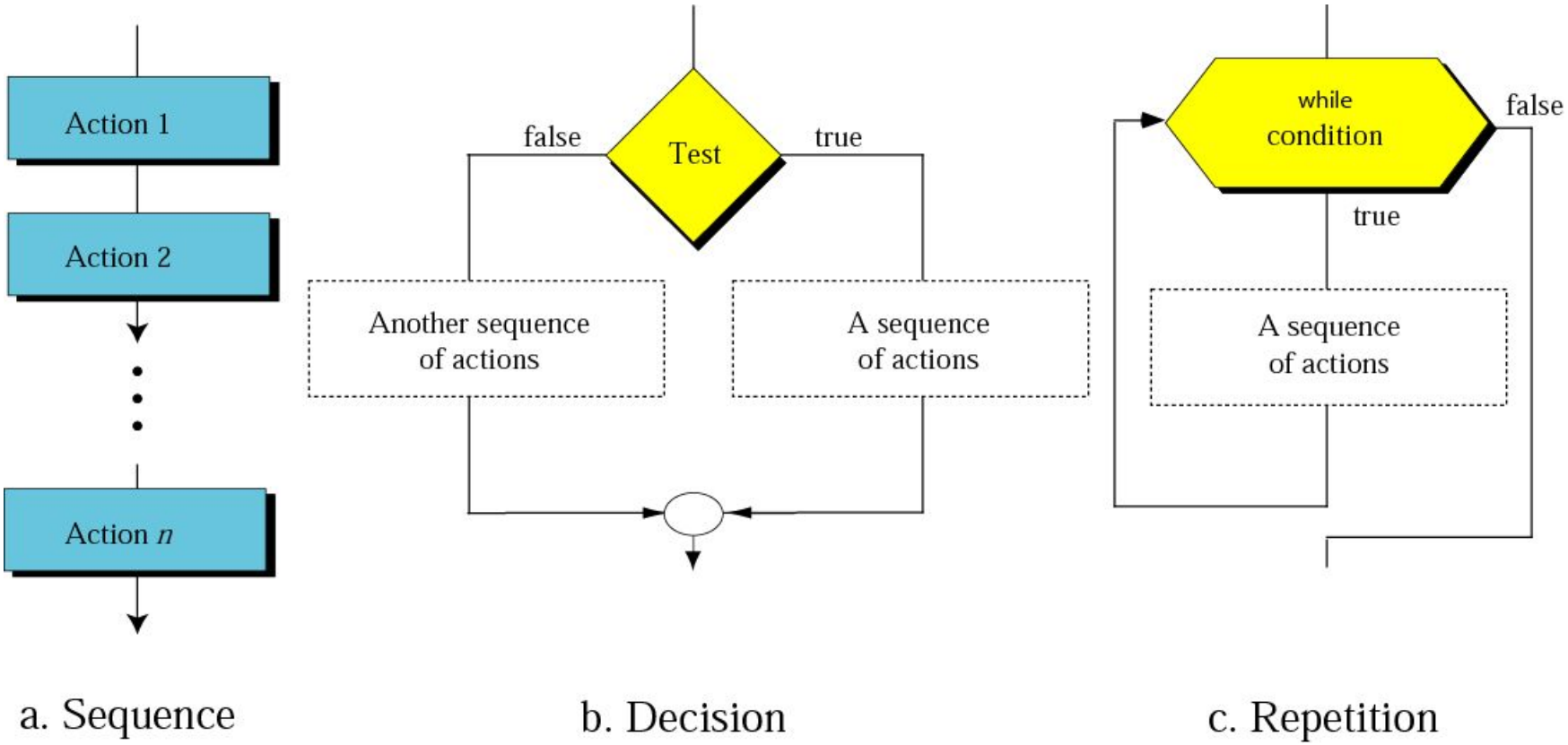
b. Decision

while a condition is true,
do action 1
do action 2
. . .
. . .
do action *n*

c. Repetition

ALGORITHM REPRESENTATION

Flowcharts for three constructs



Pseudocode for three constructs

```
action 1
action 2
   .
   .
   .
action n
```

a. Sequence

```
if (condition)
   then
      action
      action
      . . .
   else
      action
      action
      . . .
End if
```

b. Decision

```
while (condition)
      action
      action
      . . .
End while
```

c. Repetition

Example 1

Write an algorithm in pseudocode that finds the average of two numbers

Algorithm : *Average of two*

AverageOfTwo

Input: Two numbers

1. **Add the two numbers**
2. **Divide the result by 2**
3. **Return the result by step 2**

End

Example 2

Write an algorithm to change a numeric grade to a pass/no pass grade.

Algorithm : *Pass/no pass Grade*

Pass/NoPassGrade

Input: One number

1. **if (the number is greater than or equal to 70) then**
   - **1.1 Set the grade to "pass"**

   else
   - **1.2 Set the grade to "nopass"**

   End if
2. **Return the grade**

End

Example 3

Write an algorithm to change a numeric grade to a letter grade.

Algorithm : *Letter grade*

LetterGrade

Input: One number

1. **if (the number is between 90 and 100, inclusive) then**
   - **1.1 Set the grade to "A"**

   End if
2. **if (the number is between 80 and 89, inclusive) then**
   - **2.1 Set the grade to "B"**

   End if

Continues on the next slide

Algorithm : *Letter grade (continued)*

3. if (the number is between 70 and 79, inclusive) then
    3.1 Set the grade to “C”
   End if
4. if (the number is between 60 and 69, inclusive) then
    4.1 Set the grade to “D”
   End if

Continues on the next slide

Algorithm : *Letter grade (continued)*

5. **If (the number is less than 60)**
   then
   - **5.1 Set the grade to "F"**

   End if
6. **Return the grade**

   End

Example 4

Write an algorithm to find the largest of a set of numbers. You do not know the number of numbers.

Algorithm : *Find largest*

```
FindLargest
Input: A list of positive integers
1.  Set Largest to 0
2.  while (more integers)
     2.1  if (the integer is greater than Largest)
            then
              2.1.1  Set largest to the value of the integer
           End if
    End while
3.  Return Largest
    End
```

Example 5

Write an algorithm to find the largest of 1000 numbers.

Algorithm : *Find largest of 1000 numbers*

```
    FindLargest
    Input: 1000 positive integers
1.  Set Largest to 0
2.  Set Counter to 0
3.  while (Counter less than 1000)
     3.1  if (the integer is greater than Largest)
             then
               3.1.1  Set Largest to the value of the integer
            End if
      3.2  Increment Counter
    End while
4.  Return Largest
    End
```

SUBALGORITHMS

Concept of a subalgorithm

FindLargest
Input: A list of integers
1. Set Largest to 0
2. while (more integers)

   2.1 FindLarger

   End while
3. Return Largest

End

FindLarger
Input: Largest and integer
1. if (integer greater than Largest)
   then
   1.1 Set Largest to the value of integer
   End if

End

Algorithm : *Find largest*

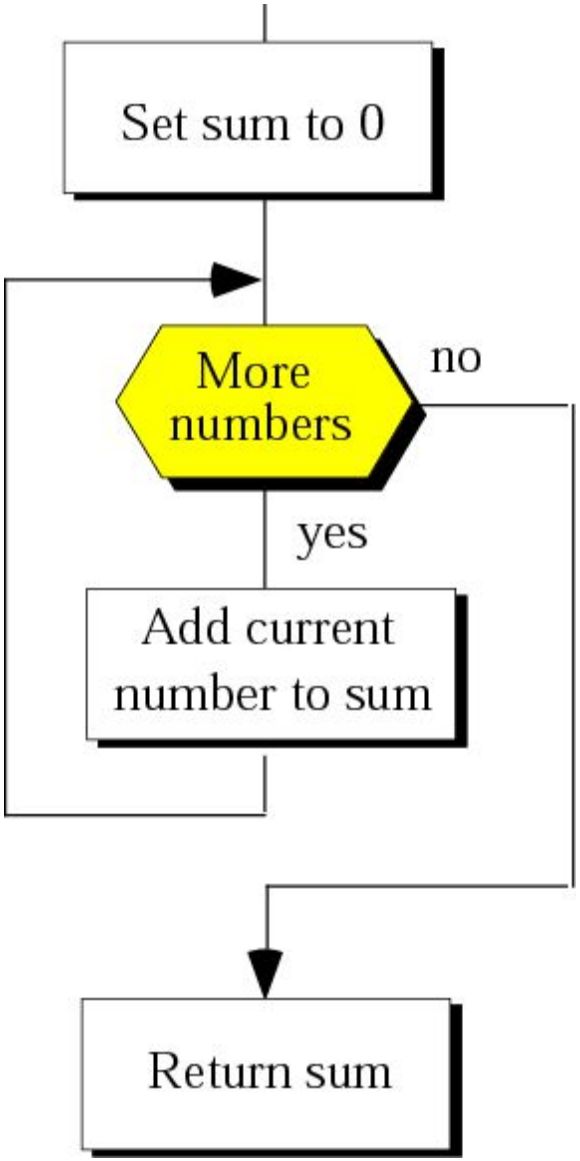
```
FindLargest
Input: A list of positive integers
1.  Set Largest to 0
2.  while (more integers)
     2.1  FindLarger
    End while
3.  Return Largest
End
```

Subalgorithm: *Find larger*

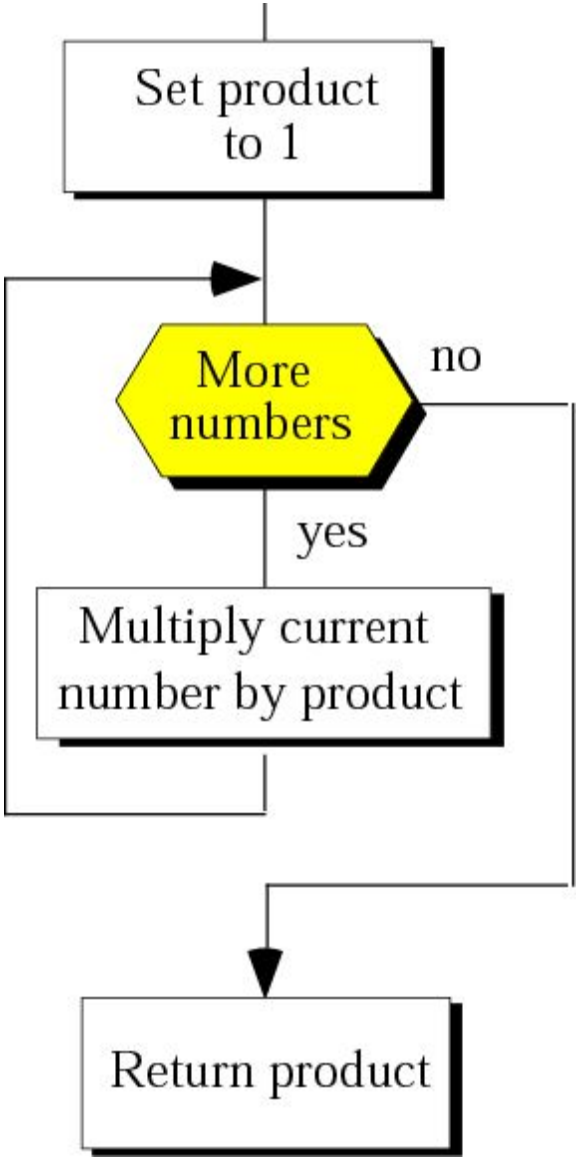
```
    FindLarger
    Input: Largest and current integer
1.  if (the integer is greater than Largest)
    then
          1.1 Set Largest to the value of the integer
    End if
    End
```

BASIC ALGORITHMS

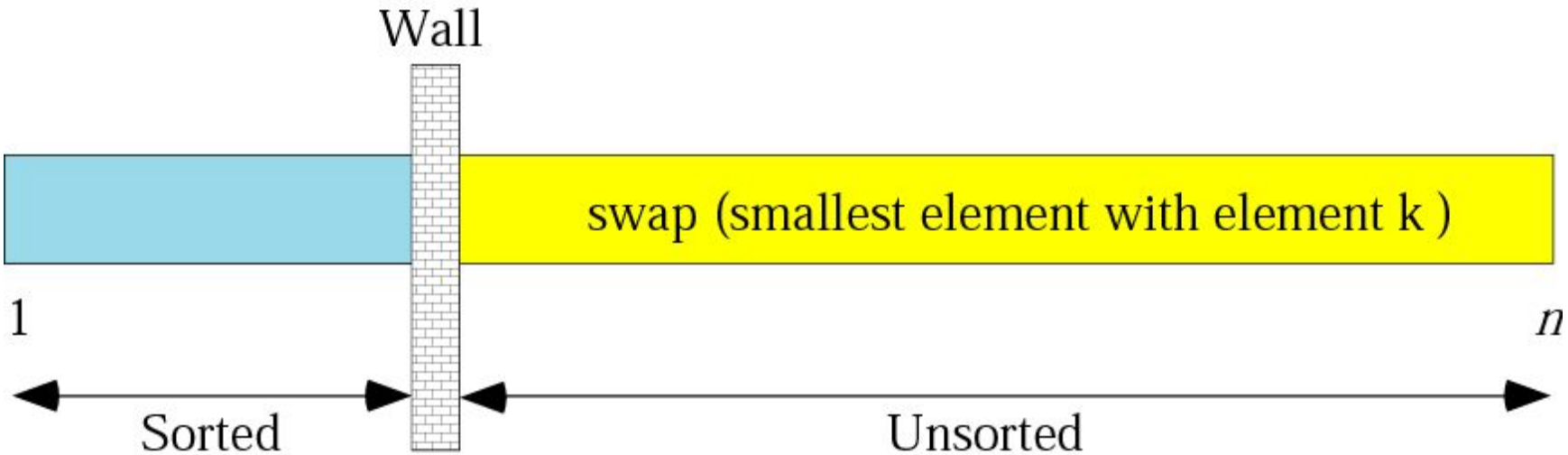
Summation


Set sum to 0
More numbers
no
yes
Add current number to sum
Return sum

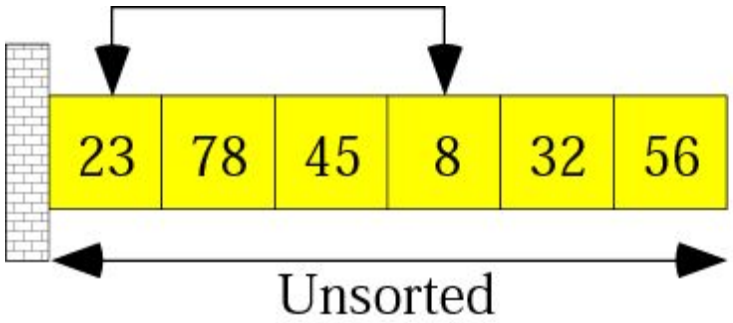
Product


Set product to 1
More numbers
no
yes
Multiply current number by product
Return product

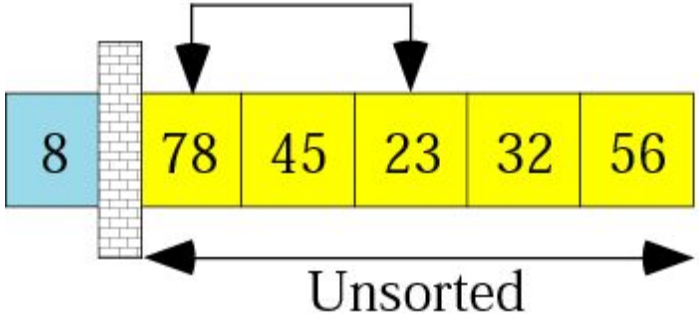
Selection sort



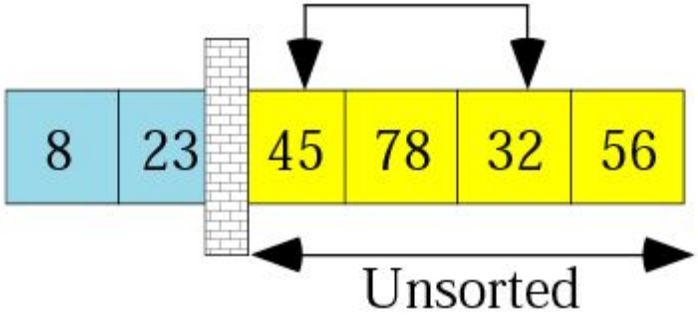
Example of selection sort



Original list

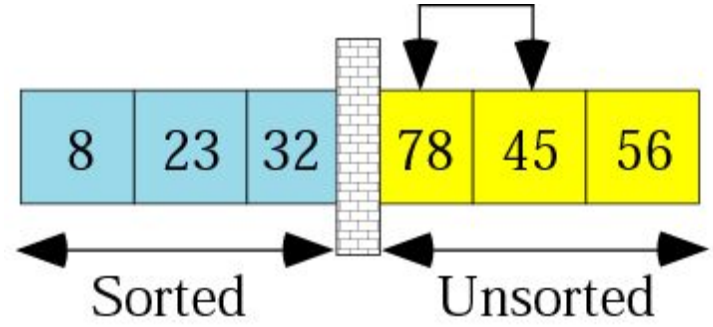


After pass 1

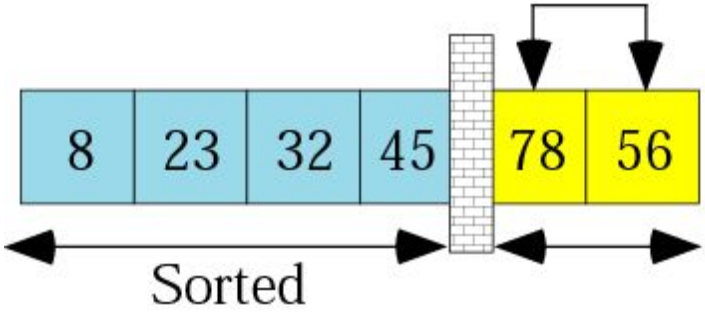


After pass 2

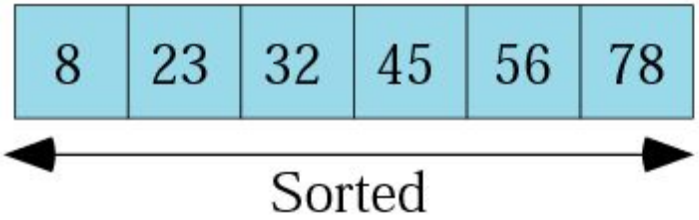
Example of selection sort



After pass 3

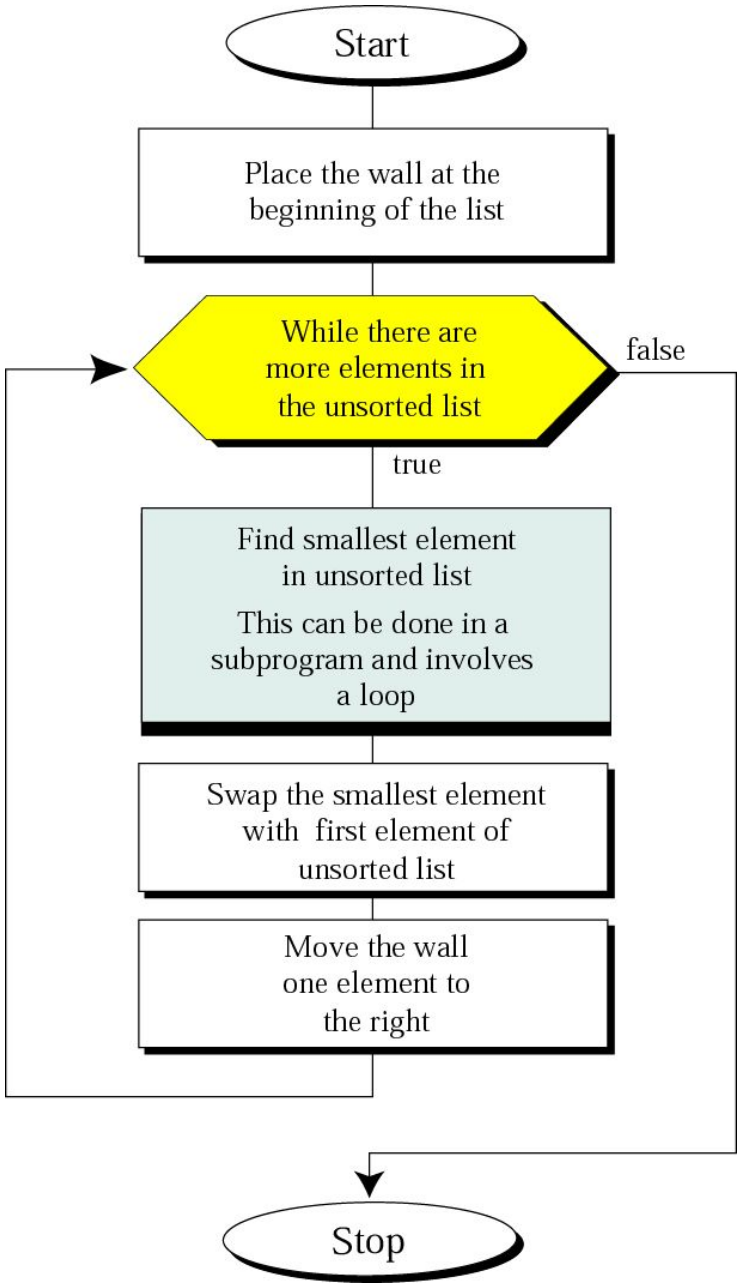


After pass 4

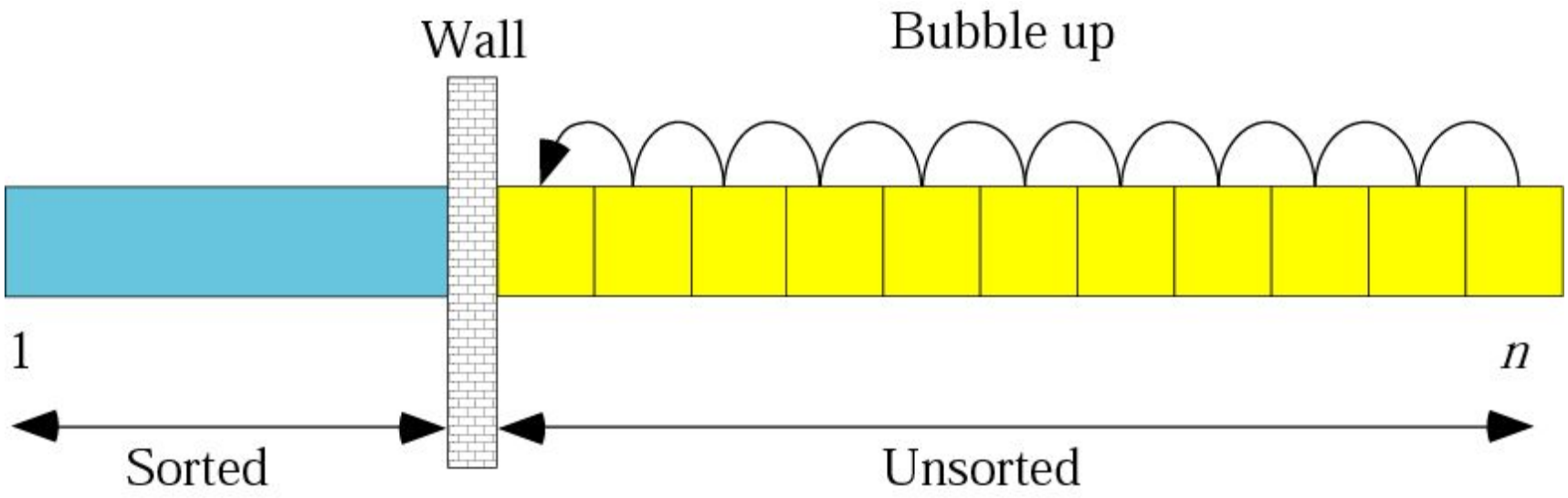


After pass 5

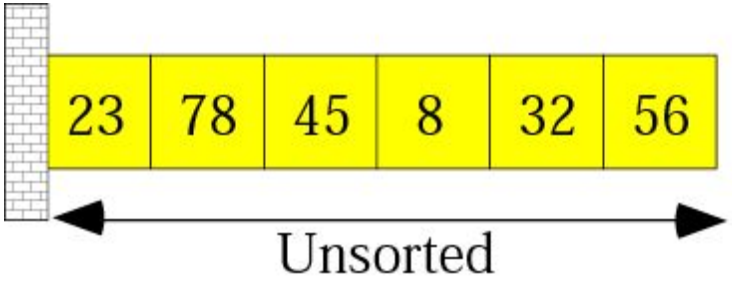
Selection sort algorithm



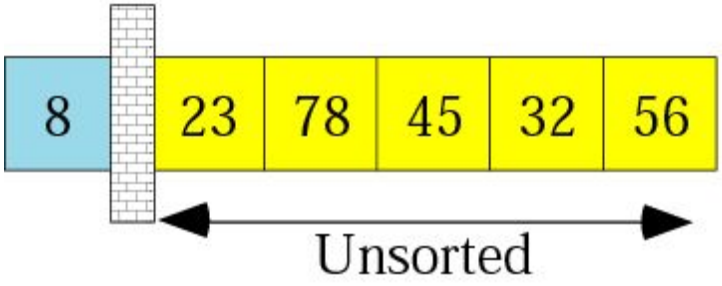
Bubble sort



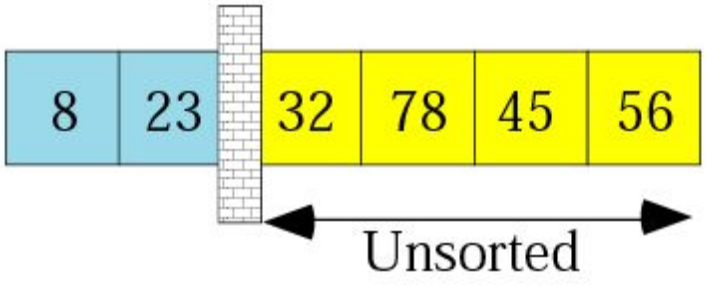
Example of bubble sort



Original list

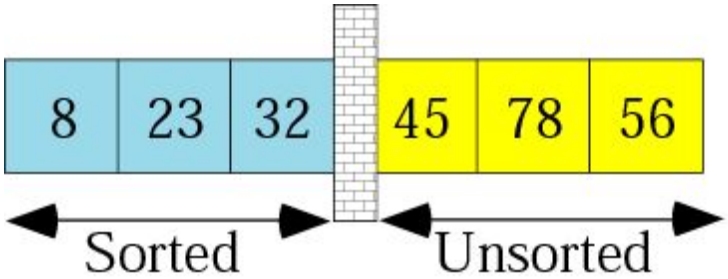


After pass 1

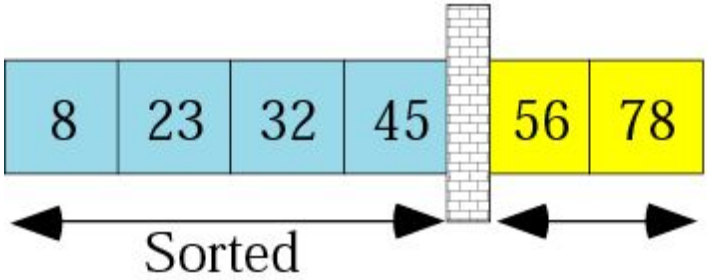


After pass 2

Example of bubble sort

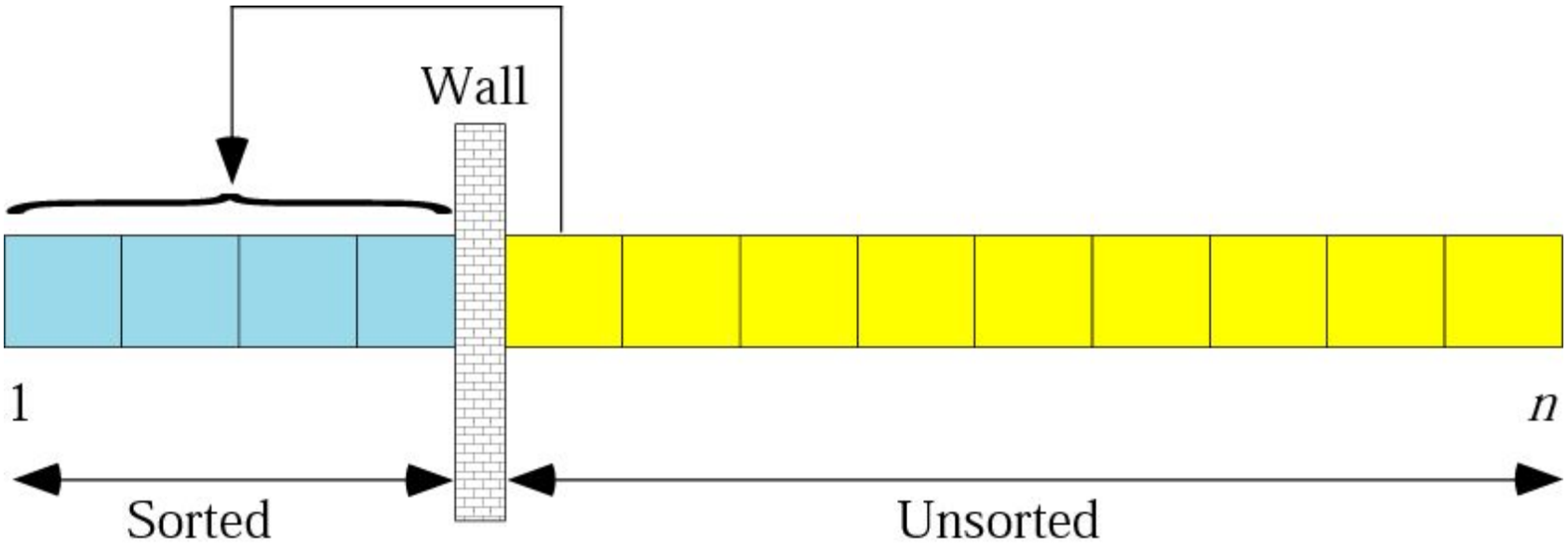


After pass 3

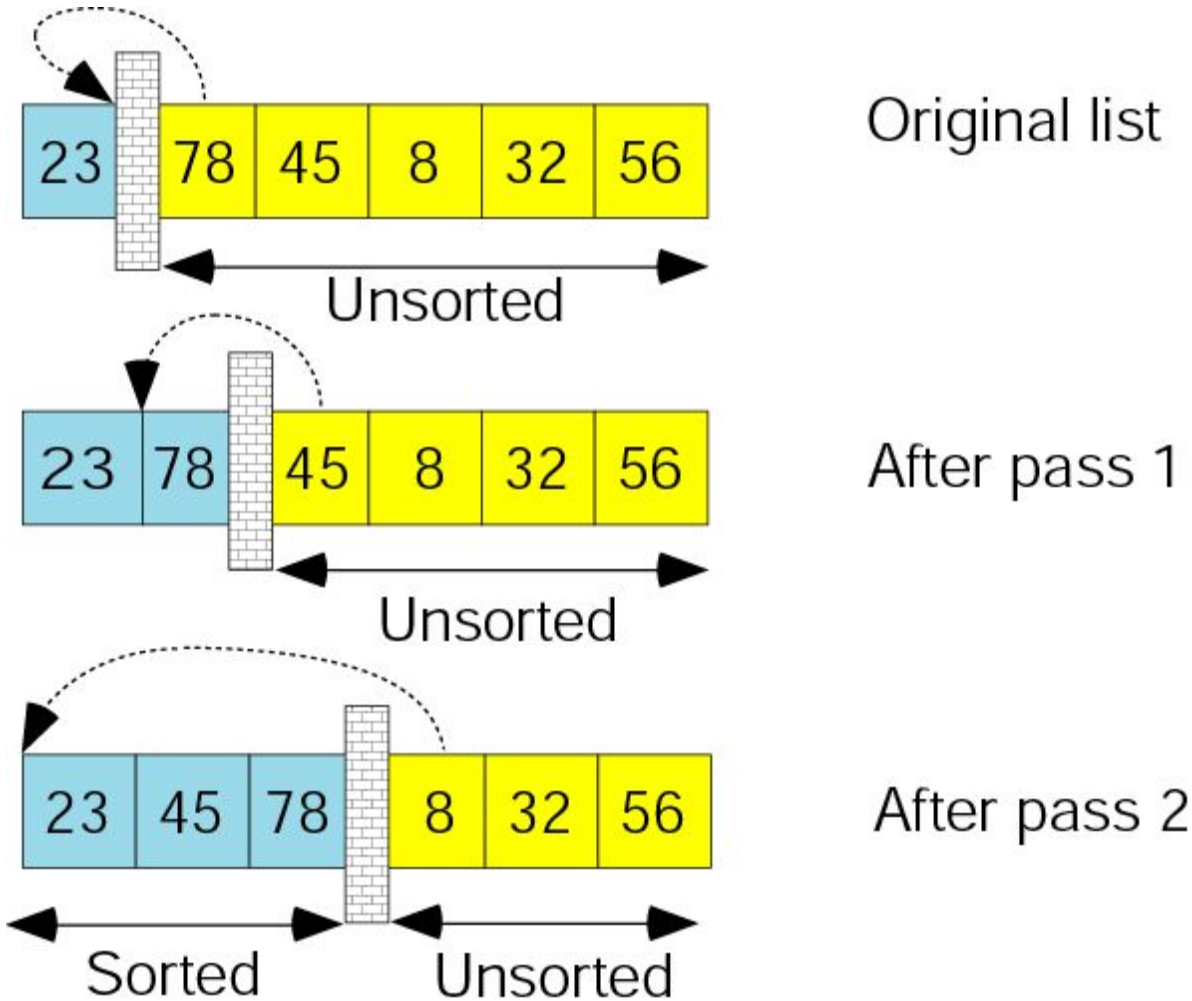


After pass 4
Sorted

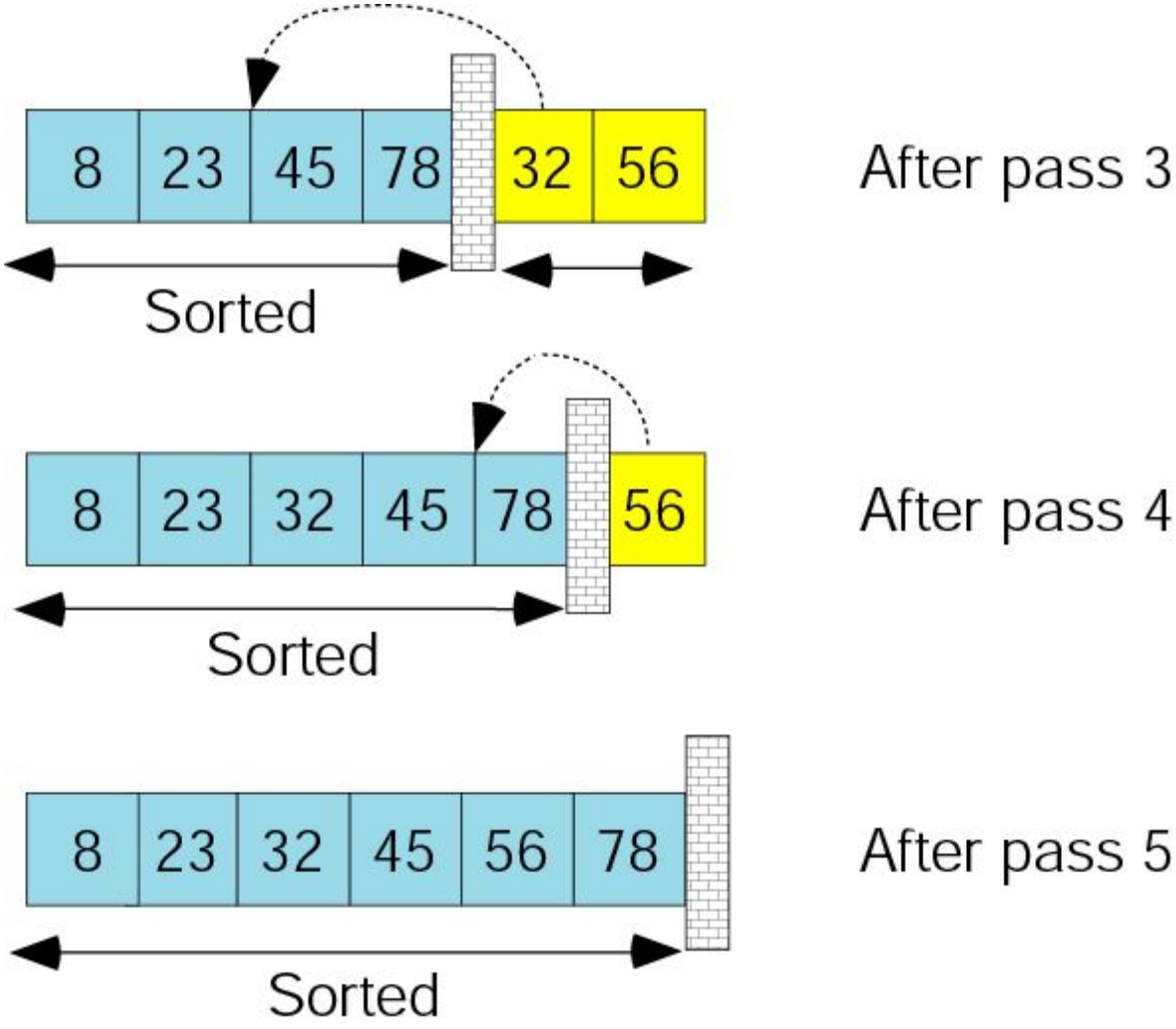
Insertion sort



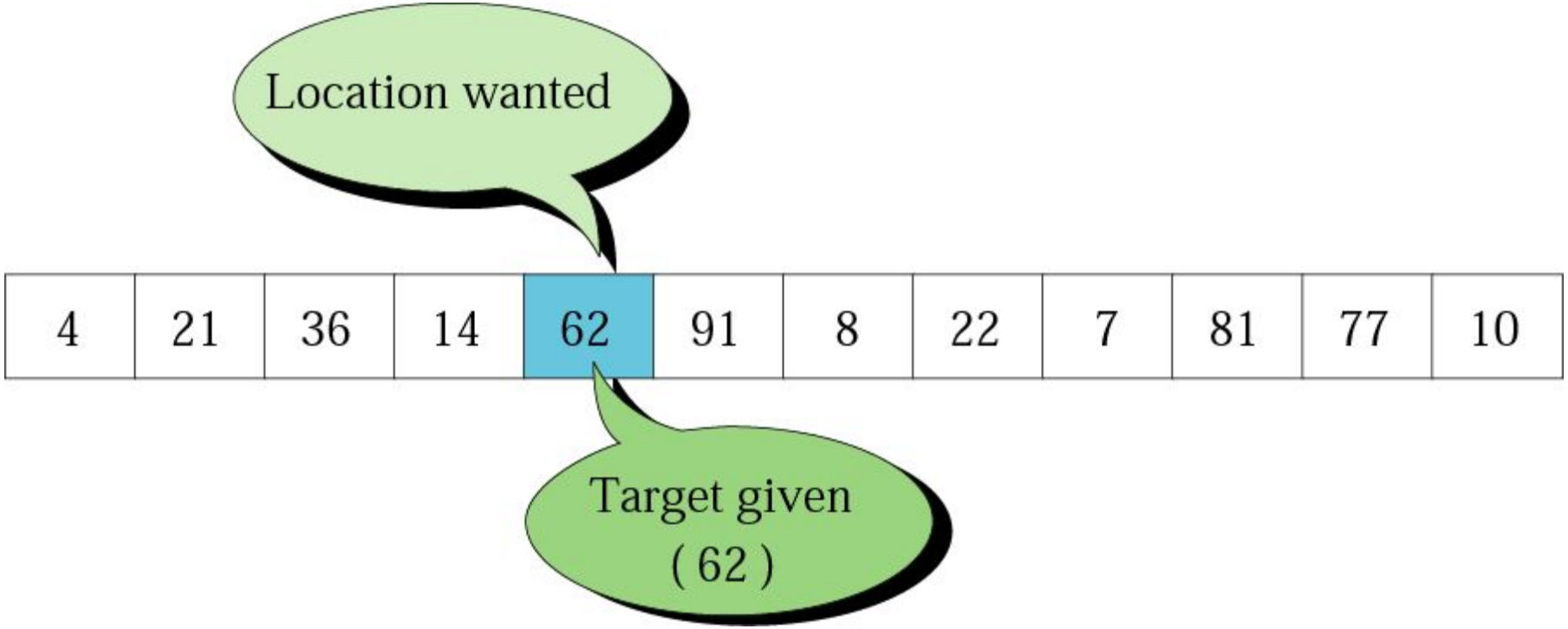
Example of insertion sort



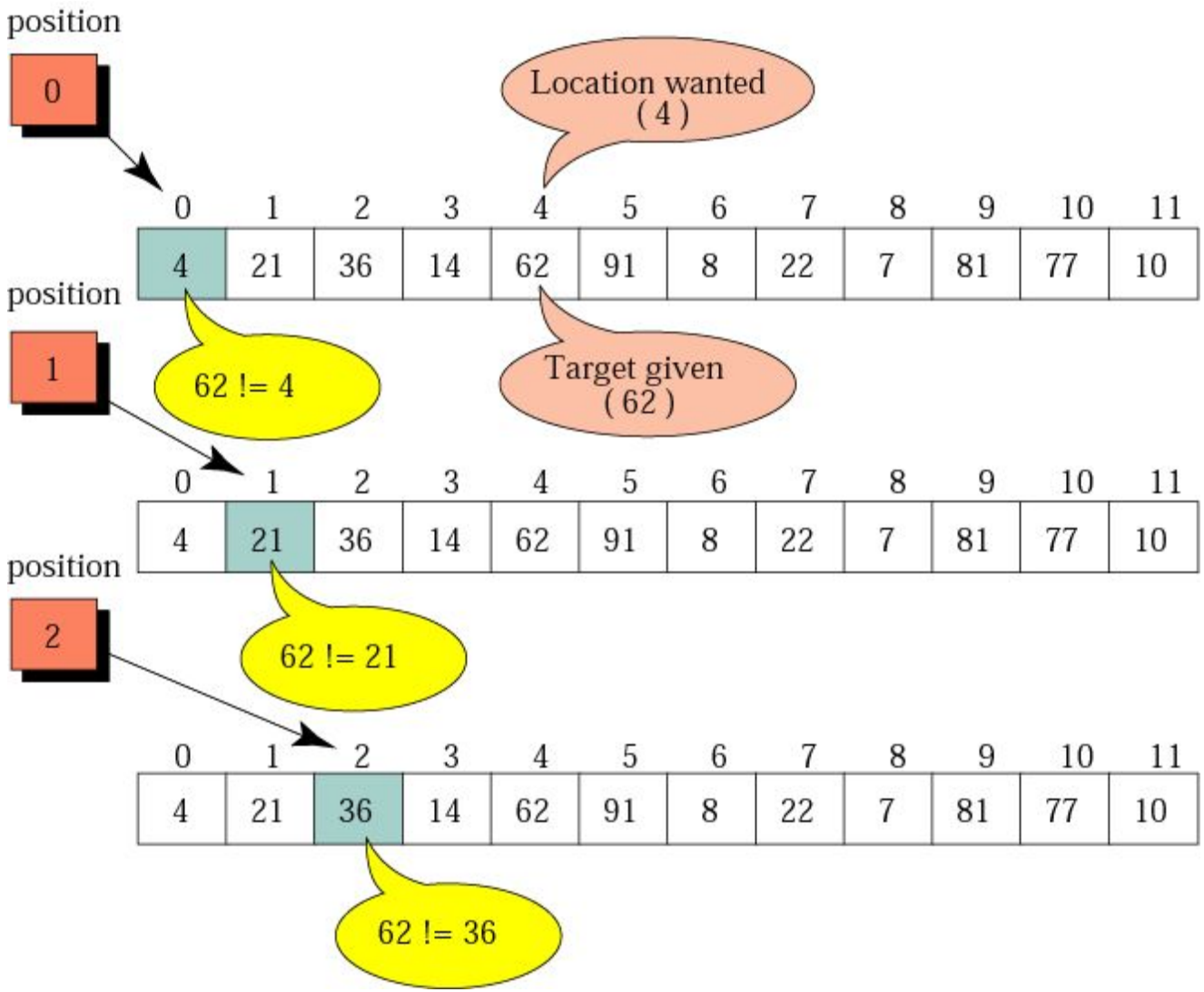
Example of insertion sort



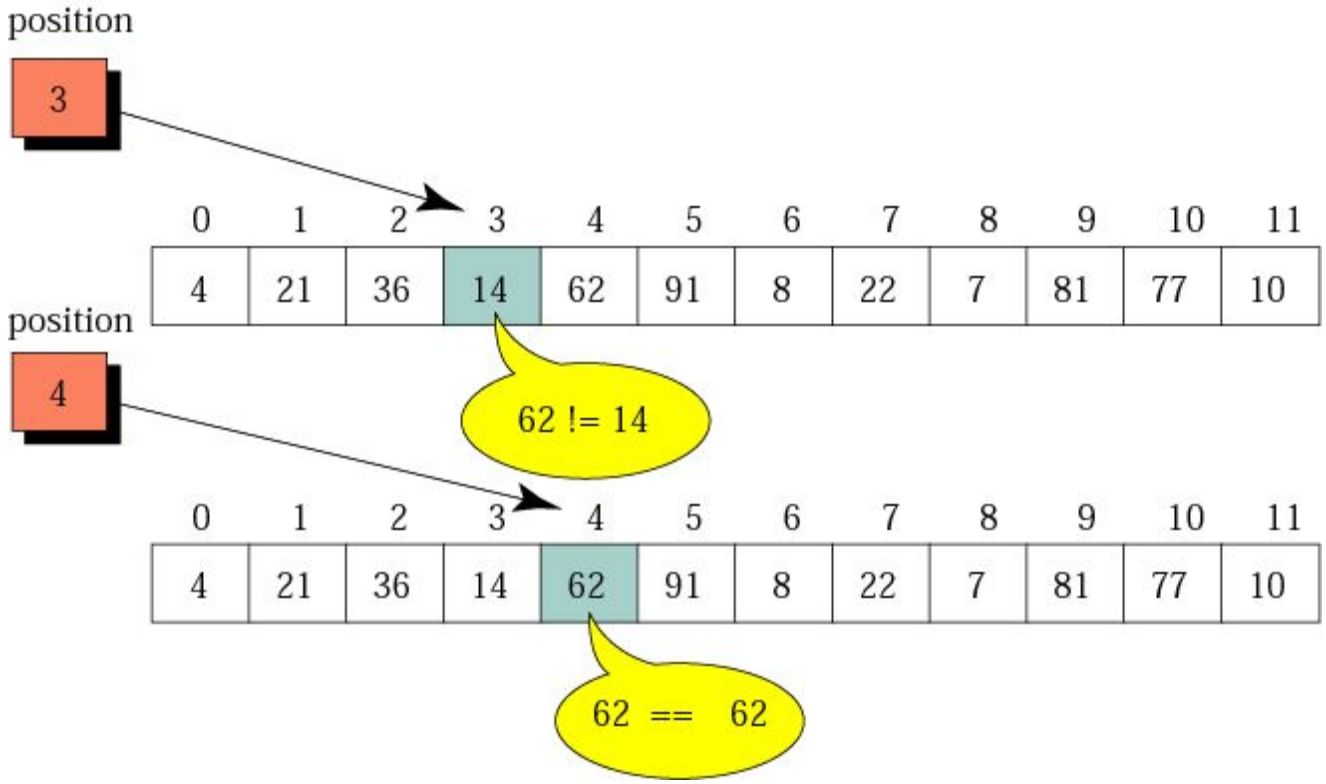
Search concept



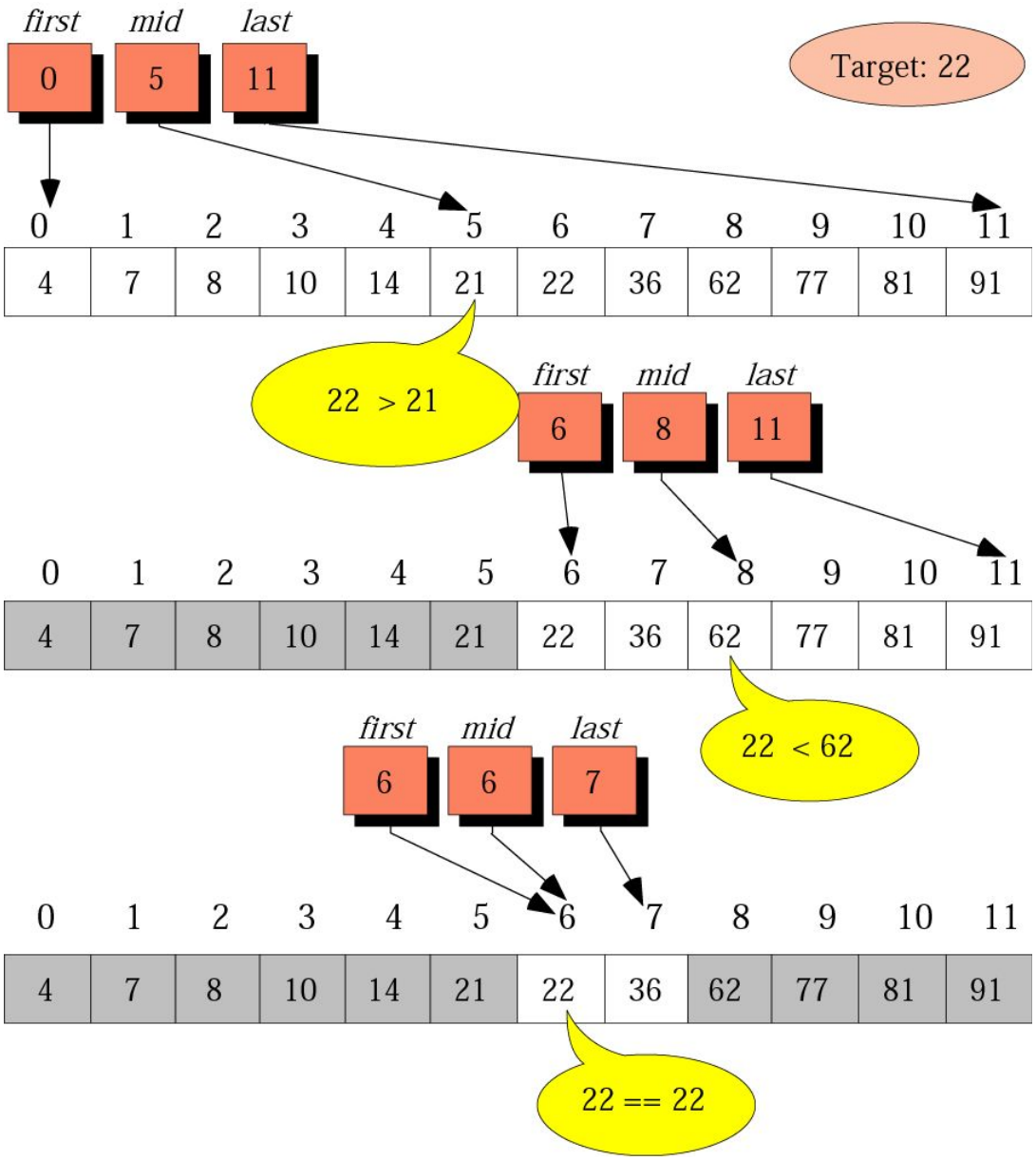
Example of a sequential sort



Example of a sequential sort



Example of a binary sort



RECURSION

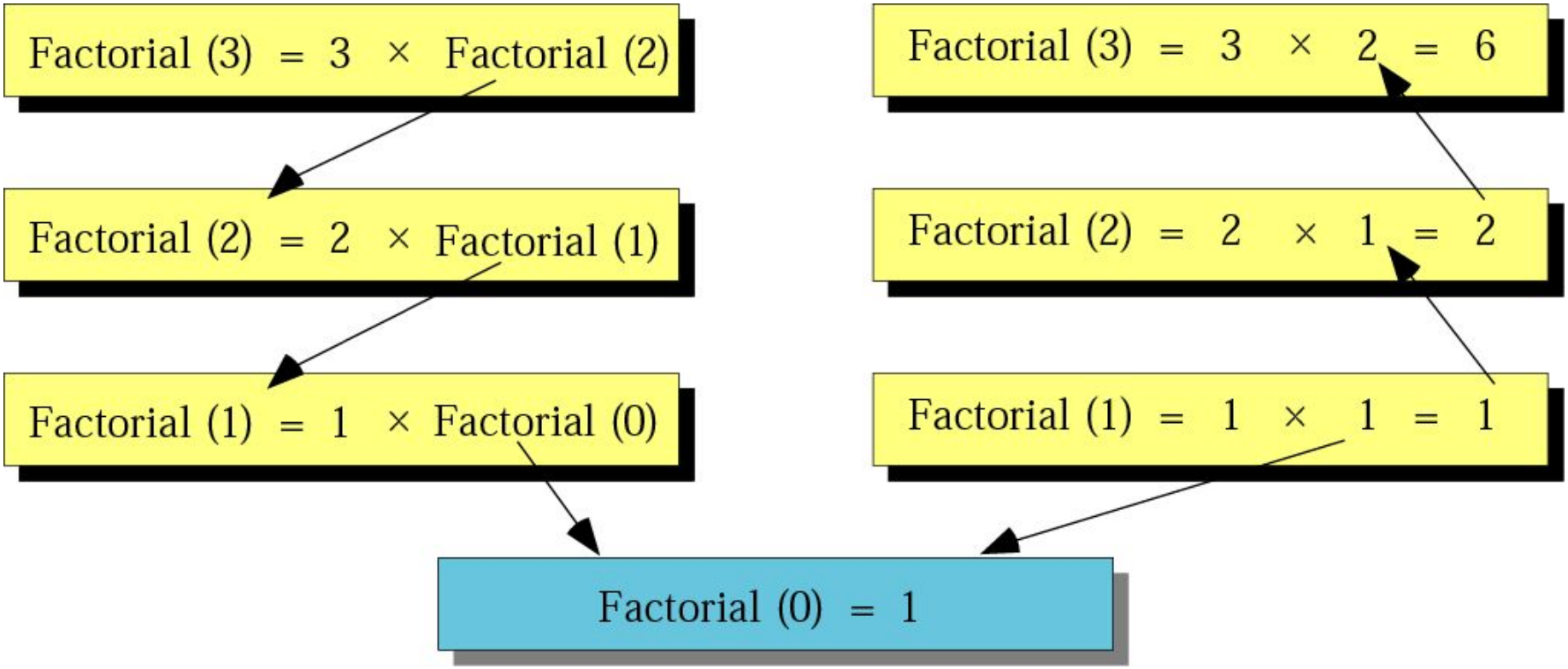
Iterative definition of factorial

$$\text{Factorial}\ (n) = \begin{cases} 1 & \text{if } n = 0 \\ n \times (n-1) \times (n-2) \times \ \ldots \ \times\ 3 \times 2 \times 1 & \text{if } n > 0 \end{cases}$$

Recursive definition of factorial

$$\text{Factorial}\ (n) = \begin{cases} 1 & \text{if } n = 0 \\ n \times \text{Factorial}\ (n-1) & \text{if } n > 0 \end{cases}$$

Tracing recursive solution to factorial problem



Algorithm : *Iterative factorial*

Factorial

Input: A positive integer num

1. **Set FactN to 1**
2. **Set i to 1**
3. **while (i is less than or equal to num)**
   - **3.1 Set FactN to FactN x I**
   - **3.2 Increment i**

   End while
4. **Return FactN**

End

Algorithm : *Recursive factorial*

```
Factorial
Input: A positive integer num
1.  if (num is equal to 0)
    then
      1.1  return 1
    else
    1.2  return num x Factorial (num – 1)
    End if
End
```